

Hotel Reservation DBMS: Normalization Report

Introduction to Normalization

When we first start thinking about building a database for a hotel, the easiest thing to do is just imagine one giant spreadsheet. We could have columns for everything: the guest's name, their phone number, the room they are staying in, what type of room it is, how much it costs, and when they are leaving. While this seems easy at first, it actually creates a ton of problems as the database grows. This is why we need database normalization.

Normalization is basically a step-by-step process we use to organize data in a database. The main goals are to reduce data redundancy (which means not storing the same piece of information in multiple places) and to improve data integrity (making sure the data makes sense and updates correctly without glitches).

The Problem with Unnormalized Data

To understand why we need this, let's look at an example. Imagine we stored everything in a single table called "AllBookings".

If John Doe books Room 101, which is a Standard room costing \$100, we put all that in one row. But what if John comes back a month later and books Room 102? We would have to type out his name, email, phone number, and address all over again in a new row. If John changes his phone number, we would have to search through the entire giant table and update every single row that belongs to him. If we miss one row, the database becomes inconsistent. This is called an update anomaly.

Also, if we wanted to add a new room category to the hotel, like a "Presidential Suite", but nobody has booked it yet, we couldn't easily add it to our giant table because a lot of the booking-related columns would be blank. That's an insertion anomaly. Normalization fixes all of these issues.

First Normal Form (1NF)

The very first step is getting our data into 1NF. The main rule here is that all the attributes (columns) must hold atomic values. This means we can't have multiple values stuffed into a single box.

For our hotel database, we made sure that every column holds just one piece of information. For example, in our Customers table, we don't have a single column called "Contact Details" holding both an email and a phone number. Instead, we split them up into separate Email and Phone columns. We also made sure that every row in every table can be uniquely identified by creating primary keys, like CustomerID in the Customers table or RoomID in the Rooms table.

Second Normal Form (2NF)

To reach 2NF, a table must first be in 1NF, and then it must not have any partial dependencies. A partial dependency happens when an attribute depends on only a part of a composite primary key, rather than the whole key.

Since most of our tables use single-column primary keys (like ReservationID or BillID), they are actually already satisfying this requirement automatically! However, conceptually, we applied this by making sure we split data into separate tables. For example, instead of mixing customer details into a table where a booking is the primary key, we moved customers into their own isolated table.

Third Normal Form (3NF)

For a table to be in 3NF, it has to be in 2NF, and it must have no transitive dependencies. A transitive dependency is when a non-key column depends on another non-key column, instead of directly on the primary key.

We applied this heavily in our database. The best example is how we handled room pricing. Originally, you might think to put BasePrice directly into the Rooms table. But the price doesn't depend on the specific room (like Room 101); it depends on the room's category (like Standard or Deluxe). If we put BasePrice in the Rooms table, the price is transitively depending on the CategoryID. To fix this and achieve 3NF, we created a separate RoomCategories table to hold the BasePrice and CategoryName.

Boyce-Codd Normal Form (BCNF)

BCNF is a slightly stronger version of 3NF. The rule for BCNF is that every determinant must be a candidate key. In simpler terms, if a column (or group of columns) determines the value of other columns, it has to be a unique key for that table. Because our database design uses straightforward single primary keys for each entity and we carefully removed transitive dependencies in the 3NF stage, our tables automatically meet the strict requirements for BCNF as well.

Table-by-Table Normalization Breakdown

Below is a brief explanation of how each table in our project fits into this normalized structure.

1. Customers Table

This table only stores information directly related to the guest. It uses CustomerID as the primary key. Attributes like FirstName, LastName, and Email are completely dependent on the customer ID and nothing else. We aren't storing any booking details here, which keeps it clean and perfectly in 3NF.

2. RoomCategories Table

This table was created specifically to solve a 3NF violation. By putting CategoryName and BasePrice here with CategoryID as the primary key, we avoid repeating the price for every single room in the hotel.

3. Rooms Table

The Rooms table uses RoomID as the primary key. It tracks the physical room number and its current status (like Available or Occupied). Instead of storing the price and category name, it just uses CategoryID as a foreign key to link back to the categories table.

4. Reservations Table

This is a crucial junction table. It uses ReservationID as the primary key. Instead of storing all the customer info and room info, it just holds the CustomerID and RoomID as foreign keys. It only contains data specific to the booking event itself, like the check-in date, check-out date, and the status of the booking.

5. Bills Table

We separated billing from the reservation to maintain a clean structure. The Bills table has its own BillID. It links to the reservation using ReservationID. This allows the system to generate a specific invoice for a booking without cluttering the reservation table with financial timestamps.

6. Payments Table

A single bill might be paid in multiple installments (for example, paying half in cash and half on a card). Because of this one-to-many relationship, putting payment details into the Bills table would violate 1NF (by having multiple payment values in one row). Creating a separate Payments table with PaymentID fixes this completely.

7. AuditLog Table

This is an independent table used purely for security and tracking. It has its own LogID. It doesn't rely on the other tables for its core structure, though it does store RecordID references. It is fully normalized for its specific purpose of tracking changes.

Final Conclusion

By going through the normalization process step-by-step, we transformed what could have been a messy, unmanageable spreadsheet into a clean, relational database system. The final design up to 3NF/BCNF ensures that we can add new customers, update room prices, and process complex bookings and payments without worrying about data anomalies or massive amounts of duplicated data. The structure is flexible, easy to query, and built to handle real-world hotel operations.